SKJS verifier · research note

Typing Promise Resolution

Native promise contracts and a sound generic Awaited<*T*>, and where TypeScript's own is unsound

Cristiano Calcagno · 7 October 2026

Outline

1. The problem: generic forwarding has no sound type
2. TypeScript's Awaited and why it is unsound
3. Native promise contracts
4. Generic Awaited<*T*>: qualified binders
5. Results: target programs, census, remaining limits

1. Generic forwarding has no sound type

```
async function wrap<T>(x: T): Promise<Awaited<T>> { return x; }
```

The verifier's trusted contracts today:

$$\%\text{Await}\% : \quad \forall T, U \le \text{Prim}.\ (\text{Promise}\langle T\rangle \mid U) \Rightarrow T \mid U$$

$$\%\text{AsyncReturn}\% : \quad \forall T, U \le \text{Prim}.\ (T \mid \text{Promise}\langle U\rangle) \Rightarrow \text{Promise}\langle T \mid U\rangle$$

T may be a promise or an object, and no type says what x becomes once a promise resolves with it. So wrap is refused.

Definitions

Prim: the primitives. Obj: every other value.

Thenable: an object with a callable then.

Resolution: a primitive or non-thenable fulfills the promise; a thenable's then is called.

Promise<*B*>: a native promise whose fulfillment values lie in *B*.

N: values that can never have a callable then.

2. TypeScript's Awaited

```ts
type Awaited<T> =
  T extends null | undefined ? T : // special case
  T extends object & { then(onfulfilled: infer F, ...args: infer _): any } ?
    F extends ((value: infer V, ...args: infer _) => any) ?
      Awaited<V> : // unwrap recursively
    never :
  T; // no declared then
```

- Decided from the *declared* type; distributes over unions; deferred on a type variable.
- Used by Promise.resolve, Promise.all, and to type await x.

Awaited is unsound on object types

Example 1 (object types are open). Accepted by tsc --strict; Node gives "hi".

```
function asRecord(x: { n: number }): { n: number } { return x; }
const t = { n: 1, then(resolve: (v: string) => void) { resolve("hi"); } };
const v = await asRecord(t); // TS: { n: number } Node: "hi"
```

Example 2 (resolution is not idempotent). Accepted by tsc --strict; Node gives 42.

```
const obj = { value: 1 };
const p: Promise<{ value: number }> = Promise.resolve(obj);
Object.assign(obj, { then: (r: (x: number) => void) => r(42) });
const first = await p;     // TS: { value: number } Node: obj
const second = await first; // TS: { value: number } Node: 42
```

Resolution can happen twice

```
const o = { n: 1 };  const p = Promise.resolve(o);  Object.assign(o, { then: (r) => r(42) });
```

Operation on p	Fulfills with
`await p`	the object o
`Promise.resolve(p)`	the object o (it returns p itself)
`Promise.all([p]) element`	the object o
`async () => p`	42
`p.then(), p.then(x => x)`	42
`p.catch(…), p.finally(…)`	42
`new Promise(r => r(p))`	42

Observing a fulfillment value is safe at any payload; resolving it again is safe only if it stays non-thenable.

3. Native promise contracts

Promise<*B*> is a native promise of the realm, with stable standard prototype, constructor, methods and species. A result that is resolved again must stay non-thenable for its whole lifetime:

> *N* = Prim | { readonly then?: Prim } | readonly unknown[]
>
> %Await% : ∀*T*, *U* ≤ *N*. (Promise<*T*> | *U*) ⇒ *T* | *U*
>
> %AsyncReturn% : ∀*T* ≤ *N*, *U* ≤ *N*. (*T* | Promise<*U*>) ⇒ Promise<*T* | *U*>
>
> %PromiseResolve% : ∀*T*, *U* ≤ *N*. (Promise<*T*> | *U*) ⇒ Promise<*T* | *U*>

await admits any promise payload *T*, and a value in *N* at its own type; async return, Promise.resolve of a plain value, catch and finally require *N*.

4. Generic Awaited<*T*>: qualified binders

Definition (qualified binder). A binder T used with Awaited comes with two companions: an observation O, what await yields, and a result R, what a fresh resolution yields.

A generic function means: it meets its signature at **every** assignment of types to (T, O, R) with

$$R \le N, \quad \text{Resolves}(T, R), \quad \text{Observes}(T, O), \quad \text{Resolves}(O, R)$$

Awaited<*T*> means R. Since R is never thenable, resolving it again changes nothing.

The relations Resolves and Observes

Rule	Resolves	Observes
stable	$A \le N$ gives $A \mapsto A$	$A \le N$ gives $A \mapsto A$
native	$A \mapsto R$ gives Promise<A> $\mapsto R$	Promise<A> $\mapsto A$
union	$A \mapsto R$, $B \mapsto S$ gives $A \mid B \mapsto R \mid S$	$A \mapsto O$, $B \mapsto Q$ gives $A \mid B \mapsto O \mid Q$
narrow	$A \le B$, $B \mapsto R$ gives $A \mapsto R$	$A \le B$, $B \mapsto O$ gives $A \mapsto O$

Relations between type meanings: resolution follows a native promise's payload until it reaches a stable type; observation reads it once. Narrow also admits wider results (number resolves to number | string); Awaited corresponds to the least.

Awaited compared with the least result

Input type	Awaited	least result *R*
primitive, null, undefined	itself	itself (stable)
Promise<number>	number	number (native, stable)
Promise<Promise<number>>	number	number (native twice)
number | Promise<readonly string[]>	number | readonly string[]	number | readonly string[] (union)
{ n: number }	{ n: number }	no descriptor: refused
PromiseLike<number>	number	no descriptor: refused
type variable *T*	deferred	qualified: descriptor at each instantiation

The least result agrees with Awaited wherever the result stays non-thenable, and refuses where Awaited is unsound.

5. Target programs

Program	Status
`accept<T>(v: Awaited<T>), a cell of Awaited<T>, wrap<number>`	verifies
`async function wrap<T>(v: T): Promise<Awaited<T>> { return await v; }`	in progress: await and return inside generic bodies
`function back<T extends number>(v: Awaited<T>): T`	open: needs a semantic decision (slide 10)
`wrap<{ n: number }>(v) // open record`	refused
`erase (v: T) => Promise<Awaited<T>> to Promise<unknown>`	refused
`async (v: Promise<{ n: number }>) => v // the 42 case`	refused

Of 888 pipeline cases, 877 meet their expected outcome; the 11 others are generic Promise cases still in progress.

Census of the SKJS corpus

SKJS corpus, before this work	Units
accepted by SKJS	2674
verified by the verifier	2466
unverified for promise reasons	6

The six promise units	With this work
queryMixedAwaitedLiteralBroadLength: await of an array	verifies (await of a stable value)
testAwaitThenable, testAwaitThenableNested: await of a PromiseLike	unchanged: needs a typed-thenable rule
createCounterService, freshReturnSnapshot: async return of a fresh object	not yet measured
testKeyof: keyof Promise<number>	unchanged: an unrelated restriction

Remaining limits

- **In progress:** await and return inside generic bodies, proof search for qualified binders, higher-order uses.
- **Awaited<*T*> is not unique:** the relations admit number with result number | string, so a result bounded by *T* needs a decision, such as taking the least result.
- **Typed thenables:** await of a PromiseLike<*T*> needs a rule for its typed then.
- **Further rules** would cover: returning a fresh object, values a Promise executor resolves, a second reaction on an already-settled promise, thenables that only resolve to themselves, and promises that can never fulfill.
- **Listed gaps:** explicit catch<string>, source Promise copies, Promise<void> reaction receivers, publishing a named generic wrapper.